

10-703 Fall 2026

Recitation 2

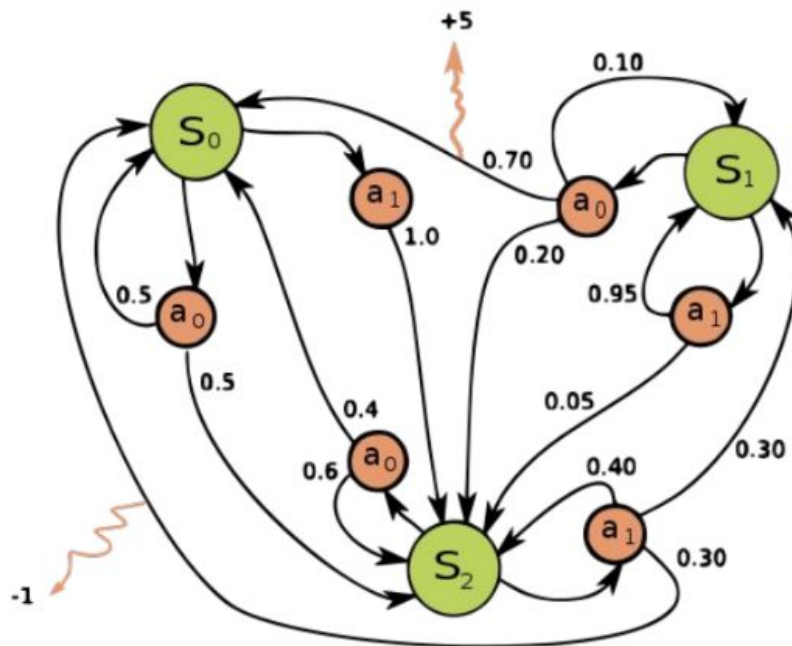
MDPs, Policy Gradient, HW1

Vansh Kapoor

Markov Decision Process (MDP)

- Simple way to represent the idea of “state”
 - The actions you take will result in new places with new reward/transition functions
- Represented oftentimes by 5 main items
 - State Space (S)
 - Action Space (A)
 - Transition Function ($T: S \times A \times S \rightarrow \text{Real}$)
 - Represents $P(s'|s, a)$
 - Reward Function ($R: S \times A \rightarrow \text{Real}$)
 - Discount Factor (γ)
- The above formulation can also represent
 - Reward functions of the form:
$$R : S \times A \times S \rightarrow \text{Real}, \text{ i.e., } R(s, a, s')$$

MDP Example



Markov Decision Process (MDP)

- Markov Property

- The idea of state encapsulating all past information (trajectory)

$$\mathbb{P} [R_{t+1} = r, S_{t+1} = s' | S_0, A_0, R_1, \dots, A_{t-1}, R_t, S_t, A_t] = \mathbb{P} [R_{t+1} = r, S_{t+1} = s' | S_t, A_t]$$

Markov Models		Do we have control over the state transitions?	
		NO	YES
Are the states completely observable?	YES	Markov Chain	MDP Markov Decision Process
	NO	HMM Hidden Markov Model	POMDP Partially Observable Markov Decision Process

MDP - What can we do?

- Goal - Learn policy $\pi: S \rightarrow A$
 - Maximize discounted reward
- Discounting rewards
 - Deals with infinite trajectories
 - Incentivizes early action
- Discount factor γ

MDP - Learning the Policy

- Value function ($V: S \rightarrow \text{Real}$)
 - Represents the estimated value of being in a state while following policy
- Q-value ($Q: S \times A \rightarrow \text{Real}$)
 - Represents the estimated value of being in a state and taking an action, then following policy

Where does the REINFORCE/A2C equation come from?

Derivatives of the policy objective

$$\max_{\theta} . U(\theta) = \mathbb{E}_{\tau \sim P_{\theta}(\tau)} [R(\tau)]$$

← We want to find the policy that maximizes expected reward when rolled out

$$\begin{aligned}\nabla_{\theta} U(\theta) &= \nabla_{\theta} \mathbb{E}_{\tau \sim P_{\theta}(\tau)} [R(\tau)] \\ &= \nabla_{\theta} \sum_{\tau} P_{\theta}(\tau) R(\tau) \\ &= \sum_{\tau} \nabla_{\theta} P_{\theta}(\tau) R(\tau) \\ &= \sum_{\tau} P_{\theta}(\tau) \frac{\nabla_{\theta} P_{\theta}(\tau)}{P_{\theta}(\tau)} R(\tau) \\ &= \sum_{\tau} P_{\theta}(\tau) \nabla_{\theta} \log P_{\theta}(\tau) R(\tau) \\ &= \mathbb{E}_{\tau \sim P_{\theta}(\tau)} [\nabla_{\theta} \log P_{\theta}(\tau) R(\tau)]\end{aligned}$$

Derivatives of the policy objective

$$\max_{\theta} . U(\theta) = \mathbb{E}_{\tau \sim P_{\theta}(\tau)} [R(\tau)]$$

← We want to find the policy that maximizes expected reward of generated trajectories

$$\begin{aligned}\nabla_{\theta} U(\theta) &= \nabla_{\theta} \mathbb{E}_{\tau \sim P_{\theta}(\tau)} [R(\tau)] \\ &= \nabla_{\theta} \sum_{\tau} P_{\theta}(\tau) R(\tau) \\ &= \sum_{\tau} \nabla_{\theta} P_{\theta}(\tau) R(\tau) \\ &= \sum_{\tau} P_{\theta}(\tau) \frac{\nabla_{\theta} P_{\theta}(\tau)}{P_{\theta}(\tau)} R(\tau) \\ &= \sum_{\tau} P_{\theta}(\tau) \nabla_{\theta} \log P_{\theta}(\tau) R(\tau) \\ &= \mathbb{E}_{\tau \sim P_{\theta}(\tau)} [\nabla_{\theta} \log P_{\theta}(\tau) R(\tau)]\end{aligned}$$

Derivatives of the policy objective

$$\max_{\theta} . U(\theta) = \mathbb{E}_{\tau \sim P_{\theta}(\tau)} [R(\tau)]$$

← We want to find the policy that maximizes expected reward of generated trajectories

$$\nabla_{\theta} U(\theta) = \nabla_{\theta} \mathbb{E}_{\tau \sim P_{\theta}(\tau)} [R(\tau)]$$

← Let's find the gradient, and run gradient Ascent!

$$= \nabla_{\theta} \sum_{\tau} P_{\theta}(\tau) R(\tau)$$

$$= \sum_{\tau} \nabla_{\theta} P_{\theta}(\tau) R(\tau)$$

$$= \sum_{\tau} P_{\theta}(\tau) \frac{\nabla_{\theta} P_{\theta}(\tau)}{P_{\theta}(\tau)} R(\tau)$$

$$= \sum_{\tau} P_{\theta}(\tau) \nabla_{\theta} \log P_{\theta}(\tau) R(\tau)$$

← $d/dx(\log f(x)) = f'(x) / f(x)$

$$= \mathbb{E}_{\tau \sim P_{\theta}(\tau)} [\nabla_{\theta} \log P_{\theta}(\tau) R(\tau)]$$

Policy Gradient Theorem

$$(\text{REINFORCE}) : \nabla_{\theta} U(\theta) = \mathbb{E}_{\tau \sim P_{\theta}(\tau)} \left(\sum_{h=0}^{H-1} \nabla_{\theta} \log(\pi_{\theta}(a_h | s_h)) \cdot R(\tau) \right).$$

$$(\text{Q-Version}) : \nabla_{\theta} U(\theta) = \mathbb{E}_{\tau \sim P_{\theta}} \left[\sum_{h=0}^{H-1} \nabla_{\theta} \log(\pi_{\theta}(a_h | s_h)) Q_h^{\pi_{\theta}}(s_h, a_h) \right].$$

Proof:

$$\begin{aligned} & \sum_{h=0}^{H-1} \nabla \log \pi_{\theta}(a_h | s_h) \cdot \left(\sum_{h=0}^{H-1} r_h \right) \\ &= \sum_{h=0}^{H-1} \nabla \log \pi_{\theta}(a_h | s_h) \cdot \left(\sum_{h=0}^{H-1} r_h \right) \\ &= \sum_{h=0}^{H-1} \nabla \log \pi_{\theta}(a_h | s_h) \cdot \left(\underbrace{\sum_{h'=0}^{h-1} r_{h'}}_A + \underbrace{\sum_{h'=h}^{H-1} r_{h'}}_B \right). \end{aligned}$$

The term A is unaffected by a_h ; when we take the expectation outside, the term A evaluates to zero. Term B in expectation is simply $Q_{\pi_{\theta}h}(s_h, a_h)$.

Policy Gradient Theorem

$$(\text{REINFORCE}) : \nabla_{\theta} U(\theta) = \mathbb{E}_{\tau \sim P_{\theta}(\tau)} \left(\sum_{h=0}^{H-1} \nabla_{\theta} \log(\pi_{\theta}(a_h | s_h)) \cdot R(\tau) \right).$$

$$(\text{Q-Version}) : \nabla_{\theta} U(\theta) = \mathbb{E}_{\tau \sim P_{\theta}} \left[\sum_{h=0}^{H-1} \nabla_{\theta} \log(\pi_{\theta}(a_h | s_h)) Q_h^{\pi_{\theta}}(s_h, a_h) \right].$$

However, since we are in a learning (RL) setup, we **cannot calculate this expectation explicitly**. We therefore use Monte Carlo estimation:

1. We first roll out N trajectories

$$\{\tau_i = (s_0^{(i)}, a_0^{(i)}, r_0^{(i)}, \dots, s_{H-1}^{(i)}, a_{H-1}^{(i)}, r_{H-1}^{(i)}, s_H^{(i)})\}_{i \in [N]}$$

following the policy π_{θ} .

2. The gradient estimate for REINFORCE is given by

$$\nabla_{\theta} U(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_h \nabla_{\theta} \log \pi_{\theta}(a_h^{(i)} | s_h^{(i)}) \cdot R(\tau_i).$$

3. Similarly, the gradient estimate for the Q-version is given by

$$\nabla_{\theta} U(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_h \nabla_{\theta} \log \pi_{\theta}(a_h^{(i)} | s_h^{(i)}) \cdot \hat{Q}_h^{(i)},$$

$$\text{where } \hat{Q}_h^{(i)} = \sum_{h'=h}^{H-1} r_{h'}^{(i)}.$$

However, the Monte-Carlo sampling method often results in a gradient estimate with **high variance**.

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

Algorithm parameter: step size $\alpha > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

 Generate an episode $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \theta)$

 Loop for each step of the episode $t = 0, 1, \dots, T - 1$:

$$G \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k \quad (G_t)$$

$$\theta \leftarrow \theta + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t, \theta)$$

For any function $b(s)$ that only depends on the state, we have

$$\begin{aligned}
 \mathbb{E}_{a \sim \pi(\cdot | s)} [b(s) \cdot \nabla_{\theta} \log \pi(a | s)] &= \sum_a b(s) \cdot \pi(a | s) \cdot \frac{1}{\pi(a | s)} \nabla_{\theta} \pi(a | s) \\
 &= b(s) \sum_a \nabla_{\theta} \pi_{\theta}(a | s) \\
 &= b(s) \nabla_{\theta} \sum_a \pi_{\theta}(a | s) \\
 &= 0.
 \end{aligned}$$

The Advantage version is obtained by subtracting the **baseline** $b(s) = V(s)$ (independent of a):

$$(\text{Advantage-Version}) \quad \nabla_{\theta} U(\theta) = \mathbb{E}_{\tau \sim P_{\theta}(\tau)} \left[\sum_{h=1}^{H-1} A_h^{\pi_{\theta}}(s_h, a_h) \nabla_{\theta} \log \pi(a_h | s_h) \right],$$

where

$$A_h^{\pi_{\theta}}(s_h, a_h) = Q_h^{\pi_{\theta}}(s_h, a_h) - V_h^{\pi_{\theta}}(s_h).$$

REINFORCE with Baseline (episodic), for estimating $\pi_{\theta} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

Input: a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$

Algorithm parameters: step sizes $\alpha^{\theta} > 0$, $\alpha^{\mathbf{w}} > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):

Generate an episode $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \theta)$

Loop for each step of the episode $t = 0, 1, \dots, T - 1$:

$$G \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k \quad (G_t)$$

$$\delta \leftarrow G - \hat{v}(S_t, \mathbf{w})$$

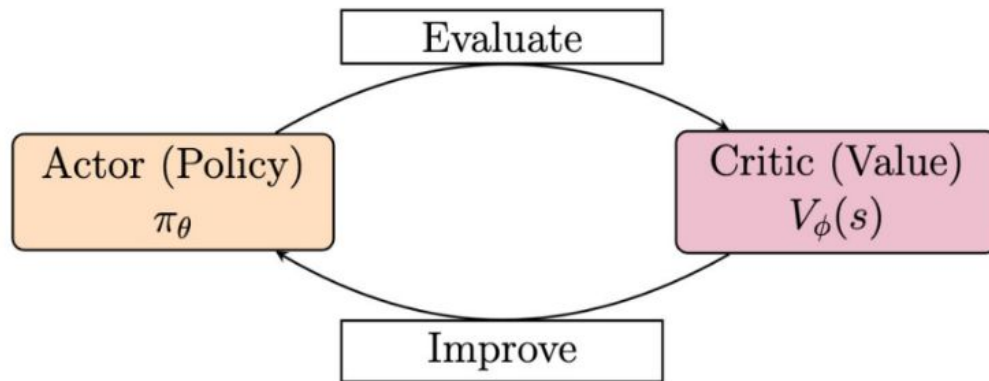
$$\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S_t, \mathbf{w})$$

$$\theta \leftarrow \theta + \alpha^{\theta} \gamma^t \delta \nabla \ln \pi(A_t | S_t, \theta)$$

To reduce variance, we replace G with $G - v(S_t, w)$ (this is called the advantage function)

Note: Actions achieving a higher score than the value function have a positive advantage, while actions with a lower score than the value function have a negative advantage. (and adding a baseline doesn't change expected gradient !)

Advantage Actor-Critic with TD(0)



We define the actor as π_θ and the critic as $V_\phi(s)$. The Advantage Actor-Critic, estimates the gradient as,

$$\nabla_\theta U(\theta) \approx \sum_{h=0}^{H-1} \nabla_\theta \log \pi(a_h | s_h) \cdot \underbrace{\left(r_h + V_\phi(s_{h+1}) - V_\phi(s_h) \right)}_{\text{Temporal Difference Error}},$$

where the **Temporal Difference (TD)** error is an estimate of $A_h^{\pi_\theta}(s_h, a_h)$.

Advantage Actor-Critic

0. Initialize policy parameters θ and critic parameters ϕ .

1. Sample trajectories $\{\tau_i = \{s_t^i, a_t^i\}_{t=0}^T\}$ by deploying the current policy $\pi_\theta(a_t | s_t)$.

2. Fit value function $V_\phi^\pi(s)$ by MC or TD estimation (update ϕ)

3. Compute action advantages $A^\pi(s_t^i, a_t^i) = R(s_t^i, a_t^i) + \gamma V_\phi^\pi(s_{t+1}^i) - V_\phi^\pi(s_t^i)$

4.
$$\nabla_\theta U(\theta) \approx \hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_\theta \log \pi_\theta(a_t^i | s_t^i) A^\pi(s_t^i, a_t^i)$$

5. $\theta \leftarrow \theta + \alpha \nabla_\theta U(\theta)$

Some Questions to Think About

- 1) Can we use policy gradient methods to solve the planning problem in MDPs, i.e., to find the optimal policy?
- 2) What could be the possible drawbacks?
(Hint: Think about the drawbacks of gradient descent.)

A2C/Reinforce/Reinforce with Baseline

Actor outputs probability of picking each action given a state

Critic outputs the predicted value for a given state

Hint 1: Actor and Critic networks should be the same except for input dimension, output dimension, and activation (probability distribution versus value function)

Q: Why don't we use the same backbone w/ different heads here?

```
49 class PolicyGradient(nn.Module):
48     def __init__(
47         self,
46         state_size,
45         action_size,
44         l1_ratio=1e-3,
43         l2_ratio=1e-3,
42         mode="REINFORCE",
41         n=0,
40         gamma=0.99,
39         device="cpu",
38     ):
37         super(PolicyGradient, self).__init__()
36
35         self.state_size = state_size
34         self.action_size = action_size
33
32         self.mode = mode
31         self.n = n
30         self.gamma = gamma
29
28         self.device = device
27
26         hidden_layer_size = 256
25
24         # actor
23         self.actor = nn.Sequential(
22             nn.Linear(state_size, hidden_layer_size),
21             nn.ReLU(),
20             nn.Linear(hidden_layer_size, action_size),
19             # BEGIN STUDENT SOLUTION
18             # END STUDENT SOLUTION
17         )
16
15         # critic
14         self.critic = nn.Sequential(
13             nn.Linear(state_size, hidden_layer_size),
12             nn.ReLU(),
11             # BEGIN STUDENT SOLUTION
10             # END STUDENT SOLUTION
9         )
8
7         # initialize networks, optimizers, move networks to device
6         # BEGIN STUDENT SOLUTION
5         pass
4         # END STUDENT SOLUTION
```

Generate Episode: run the actor/policy, and save the list of states, actions, and rewards generated (needed for training)

Compute goal / advantage: Watch out for off-by-one errors in the n-step bootstrap!

Evaluate Policy: run the actor/policy, and save the undiscounted sum of rewards/length of trajectory (needed for plot creation/testing)

Train: you should have different cases for reinforce, reinforce with baseline, and A2C

Define the losses for each of these and run training (optimizer.zero_grad(), loss.backward(), optimizer.step()) etc)

Run: Within each episode, call env.reset() and step through for max_steps. If terminated, then break and move on to the next episode (make sure to update the states, actions and rewards you've seen in that episode)

```
def get_action(self, state, stochastic):
    # if stochastic, sample using the action probabilities, else get the argmax
    # BEGIN STUDENT SOLUTION
    pass
    # END STUDENT SOLUTION

def calculate_n_step_bootstrap(self, rewards_tensor, values):
    # calculate n step bootstrap
    # BEGIN STUDENT SOLUTION
    pass
    # END STUDENT SOLUTION

def train(self, states=None, actions=None, rewards=None):
    if actions is None and rewards is None and (
        states is None or isinstance(states, bool)
    ):
        return super().train(True if states is None else states)

    # train the agent using states, actions, and rewards
    # BEGIN STUDENT SOLUTION
    pass
    # END STUDENT SOLUTION

def run(self, env, max_steps, num_episodes, train):
    total_rewards = []

    # run the agent through the environment num_episodes times for at most max steps
    # BEGIN STUDENT SOLUTION
    pass
    # END STUDENT SOLUTION
    return total_rewards
```

Questions?

References

Some of the slides are heavily inspired by the notes from the class

17-740: Algorithmic Foundations of Interactive Learning,

and some are directly taken from Sutton & Barto's [Reinforcement Learning: An Introduction](#)